



Fifo Server

FIFO Server: procesarea datelor în timp real

Jan 9, 2025

Prezentare generală

Despre Fifo Server

Proiectul **FIFO Server** implementează un sistem client-server în care comunicația între componente se realizează prin intermediul fișierelor speciale de tip FIFO, oferind un mecanism eficient și sincronizat pentru procesarea cererilor. Serverul așteaptă cereri de la clienți, care solicită informații din paginile de manual ale diferitelor comenzi shell. Fiecare cerere conține PID-ul clientului și numele comenzii dorite. După procesarea fiecărei solicitări, serverul generează un răspuns, care este trimis înapoi clientului printr-un fișier FIFO unic, personalizat, ce include PID-ul clientului. Astfel, proiectul permite gestionarea concurentă a cererilor, cu un răspuns rapid și izolat pentru fiecare client, asigurând o comunicare fiabilă și scalabilă.

Despre noi

Noi, **Leuștean Ștefan**, **Minea Teodor Ștefan** și **Pupezescu Alina**, suntem studenți în anul 1 la Facultatea de Matematică și Informatică, specializarea Informatică, grupa 141. Am ales tema **Fifo Server** pentru a învăța și aplica concepte de scripting în Bash, utilizând fișiere FIFO pentru comunicarea între procese. Această temă ne-a oferit ocazia de a explora un aspect important al programării de sistem, adică gestionarea concurentă a cererilor într-un mediu controlat,

folosind scripturi shell pentru a implementa soluții eficiente de comunicare între client și server.

Fifo Server

Proiectul **FIFO Server** implementează un sistem simplu de comunicare între client și server folosind fișiere FIFO, într-un mediu controlat prin scripturi Bash. Sistemul este format din cinci fișiere principale, fiecare având un rol specific în procesul de gestionare a cererilor și răspunsurilor între client și server.

Fișierul **config.sh** este folosit pentru a stabili variabile esențiale, cum ar fi calea către server și fișierele FIFO personalizate pentru fiecare client. De exemplu, **SERVER_PATH** definește calea unde serverul va căuta fișierul FIFO de comunicare:

```
bash
```

```
SERVER_PATH="/home/teominea/Desktop/facultate/ITBI/proiect/$SERVER_NAME"
```

Fișierul **run_server.sh** este responsabil cu pornirea și oprirea serverului. Acesta folosește comenzi pentru a verifica dacă serverul este deja activ și, dacă nu, pornește procesele relevante, cum ar fi **fifo_server_side.sh** (care gestionează cererile clienților) și **verify_if_suspended.sh** (care monitorizează procesele clienților). De exemplu, pentru a porni serverul:

```
bash
```

```
nohup ./fifo_server_side.sh > /dev/null 2>&1 &
```

Scriptul principal care gestionează cererile clienților este **fifo_server_side.sh**. Când serverul primește o cerere de tipul BEGIN-REQ [client-pid: command-name] END-REQ din fișierul FIFO, procesează comanda solicitată și trimite rezultatul într-un fișier FIFO personalizat pentru client. De exemplu, dacă comanda solicitată există, serverul va returna conținutul paginii de manual:

```
bash
```

```
man ${COMMAND_NAME} > $personalFifo
```

În schimb, dacă comanda nu există, serverul va răspunde cu un mesaj de eroare:

```
bash
echo "Command doesn't exist $COMMAND_NAME" > $personalFifo
```

Scriptul **verify_if_suspended.sh** se asigură că procesele clienților care au fost suspendate sunt terminate corect. Acesta verifică dacă procesele sunt suspendate (starea T), iar în caz afirmativ, le închide și șterge fișierele FIFO aferente:

```
bash
if ps -o stat= -p ${PID_OF_CLIENT} | grep -q 'T'; then
    rm $myFifo
    kill -9 ${PID_OF_CLIENT}
fi
```

În final, **fifo_client_side.sh** este scriptul care trimite cereri către server. Clientul creează o cerere de tipul BEGIN-REQ și o trimite serverului prin fișierul FIFO principal, iar apoi așteaptă răspunsul într-un fișier FIFO personalizat. După ce răspunsul este primit, clientul îl vizualizează cu comanda less:

```
bash
less -f $myFifo
```

Acest proiect ne-a permis să înțelegem mai bine cum funcționează comunicarea între procese în sistemele Unix, utilizând fișiere FIFO și scripturi Bash pentru a implementa un server simplu, dar eficient.

Expresiile Regulate în Procesarea Cererilor

Un alt aspect esențial al implementării FIFO Server este utilizarea **expresiilor regulate** (regex) pentru procesarea cererilor. În **fifo_server_side.sh**, serverul așteaptă cereri de la clienți într-un format bine definit, care poate fi validat printr-o expresie regulată. Cererea clientului este structurată astfel:

```
bash
```

```
BEGIN-REQ [client-pid: command-name] END-REQ
```

Pentru a valida aceste cereri și a extrage informațiile relevante (PID-ul clientului și comanda solicitată), folosim următoarea expresie regulată:

```
bash
```

```
if [[ "$request" =~ BEGIN-REQ\ \([(0-9)+):\ ([a-zA-Z0-9_-]+\)\ END-REQ ]]; then  
  CLIENT_PID="${BASH_REMATCH[1]}"  
  COMMAND_NAME="${BASH_REMATCH[2]}"
```

Expresia regulată **BEGIN-REQ\ \([(0-9)+):\ ([a-zA-Z0-9_-]+\)\ END-REQ** este folosită pentru a căuta și extrage două elemente cheie din cererea clientului:

1. **PID-ul clientului**: Reprezintă identificatorul procesului care a trimis cererea. Este capturat de grupul `[(0-9)+)`, care permite doar cifre.
2. **Numele comenzii**: Reprezintă comanda shell pentru care clientul dorește informații. Este capturat de grupul `[a-zA-Z0-9_-]+\)`, care permite litere, cifre, cratime și underscore.

Expresia regulată caută exact acest șablon în cererea clientului și, dacă îl găsește, folosește variabila **BASH_REMATCH** pentru a salva valorile capturate într-o variabilă. În cazul nostru, **CLIENT_PID** va conține PID-ul clientului, iar **COMMAND_NAME** va conține numele comenzii pentru care clientul cere informațiile. Aceste date sunt apoi folosite pentru a procesa cererea și a returna rezultatele corecte.

Dificultăți Întâmpinate în Crearea Proiectului

- Rularea scriptului în background fără a fi oprit la închiderea terminalului

A fost necesar să ne asigurăm că scripturile rulează în background și continuă să funcționeze chiar și după ce terminalul este închis. Am folosit comanda `nohup` pentru a preveni oprirea acestora:

```
bash
```

```
nohup ./fifo_server_side.sh > /dev/null 2>&1 &
```

- Ștergerea fișierelor temporare la întreruperea scriptului client

În cazul în care clientul întrerupe execuția (de exemplu, apăsând Ctrl+C), trebuia să ștergem fișierele FIFO personalizate pentru a evita acumularea de fișiere inutile pe sistem. Acest lucru a fost realizat prin utilizarea funcției trap:

```
bash

trap cleanup SIGHUP SIGINT SIGTERM EXIT
cleanup() {
    if [ -e "$myFifo" ]; then
        rm "$myFifo"
    fi
}
```

- Gestionarea cazului în care comanda introdusă nu există

În cazul în care un client cere informații despre o comandă care nu există, serverul returnează un mesaj de eroare, indicând că aceasta nu poate fi găsită:

```
bash

echo "Command doesn't exist $COMMAND_NAME" > $personalFifo
```

- Verificarea existenței unei instanțe active a serverului

Am implementat un mecanism care verifică dacă serverul este deja activ, pentru a preveni lansarea unei noi instanțe atunci când una este deja în execuție. Acest lucru a fost realizat cu ajutorul comenzii pgrep:

```
bash

if [[ $(pgrep -f "fifo_server_side.sh" | wc -l) -gt 0 ]]; then
    echo "Server is already running!"
else
    nohup ./fifo_server_side.sh > /dev/null 2>&1 &
fi
```

Contribuțiile fiecăruia

Membru	Responsabilități	Specificații
Leuștean Ștefan	☐ A gestionat partea server a proiectului	☐ Transmiterea datelor corecte către client ☐ Crearea unui FIFO nou la fiecare rulare ☐ Comunicarea server-client prin FIFO ☐ Verificarea stării serverului înainte de pornire
Minea Teodor Ștefan	☐ Gestionarea pornirii, opririi serverului și a situațiilor neprevăzute	☐ Gestionarea comenzilor inexistente ☐ Ștergerea fișierelor răspunsurilor ☐ Cleanup și crearea FIFO-uri noi ☐ Pornirea și oprirea serverului
Pupezescu Alina-Elena	☐ A implementat partea client a proiectului	☐ Transmiterea cererii corecte ☐ Validarea numelui cererii ☐ Crearea și ștergerea fișierului ☐ Afișarea manualului

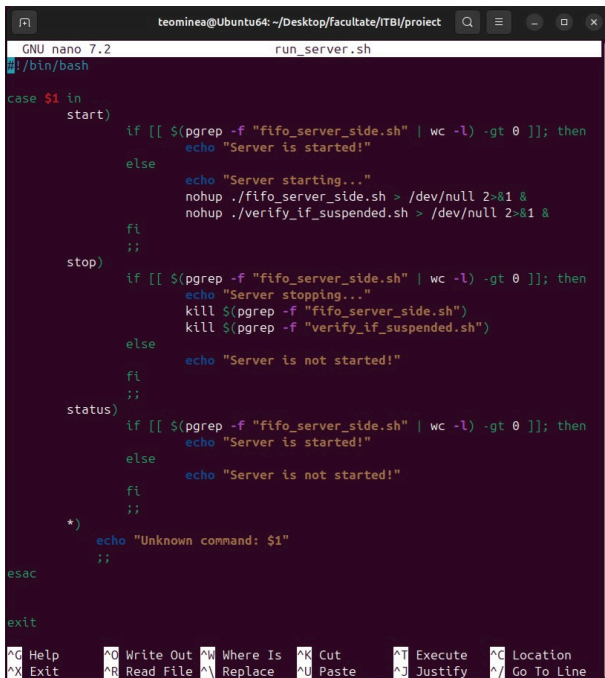
Imagini

```
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./run_server.sh start
Server starting...
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./run_server.sh status
Server is started!
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./fifo_client_side.sh pwd
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./fifo_client_side.sh ls

[1]+  Stopped                  ./fifo_client_side.sh ls
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./fifo_client_side.sh man
[1]  Killed                    ./fifo_client_side.sh ls

[2]+  Stopped                  ./fifo_client_side.sh man
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./fifo_client_side.sh ls
[2]+  Killed                    ./fifo_client_side.sh man
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./run_server.sh status
Server is started!
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./fifo_client_side.sh chmod
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./run_server.sh stop
Server stopping...
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$ ./run_server.sh status
Server is not started!
teominea@Ubuntu64:~/Desktop/facultate/ITBI/proiect$
```

Implementarea funcționalităților start, stop și tratarea cazurilor speciale

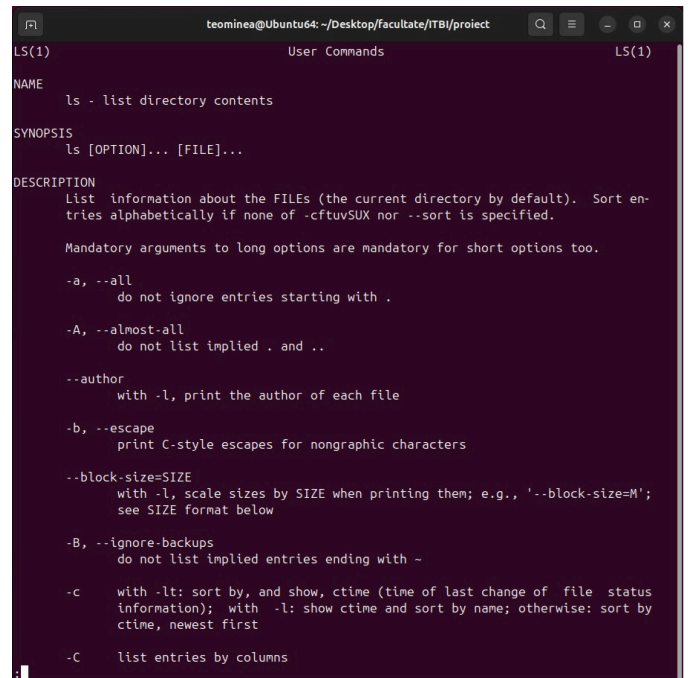


```
GNU nano 7.2 run_server.sh
#!/bin/bash

case $1 in
    start)
        if [[ $(pgrep -f "fifo_server_side.sh" | wc -l) -gt 0 ]]; then
            echo "Server is started!"
        else
            echo "Server starting..."
            nohup ./fifo_server_side.sh > /dev/null 2>&1 &
            nohup ./verify_if_suspended.sh > /dev/null 2>&1 &
        fi
        ;;
    stop)
        if [[ $(pgrep -f "fifo_server_side.sh" | wc -l) -gt 0 ]]; then
            echo "Server stopping..."
            kill $(pgrep -f "fifo_server_side.sh")
            kill $(pgrep -f "verify_if_suspended.sh")
        else
            echo "Server is not started!"
        fi
        ;;
    status)
        if [[ $(pgrep -f "fifo_server_side.sh" | wc -l) -gt 0 ]]; then
            echo "Server is started!"
        else
            echo "Server is not started!"
        fi
        ;;
    *)
        echo "Unknown command: $1"
        ;;
esac

exit
```

Scriptul de rulare a serverului



```
LS(1) User Commands LS(1)

NAME
ls - list directory contents

SYNOPSIS
ls [OPTION]... [FILE]...

DESCRIPTION
List information about the FILES (the current directory by default). Sort
entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.

-a, --all
do not ignore entries starting with .

-A, --almost-all
do not list implied . and ..

--author
with -l, print the author of each file

-b, --escape
print C-style escapes for nongraphic characters

--block-size=SIZE
with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M';
see SIZE format below

-B, --ignore-backups
do not list implied entries ending with ~

-c with -lt: sort by, and show, ctime (time of last change of file status
information); with -l: show ctime and sort by name; otherwise: sort by
ctime, newest first

-C list entries by columns
```

Deschiderea fișierului solicitat de client

Cerințe de Funcționare

1. **Sistem de operare Linux** – Proiectul funcționează pe orice distribuție Linux, deoarece utilizează comenzi și funcționalități native ale sistemului de operare, precum manipularea fișierelor FIFO și gestionarea proceselor.
2. **Cunoștințe de bază în terminalul Linux** – Utilizarea terminalului pentru rularea scripturilor, gestionarea fișierelor și monitorizarea proceselor este esențială.
3. **Permișiuni de execuție** – Fișierele de script trebuie să fie setate ca executabile (`chmod +x <file_name>`).
4. **Bash shell** – Toate scripturile sunt scrise în Bash, așadar, este necesar să fie disponibil un shell Bash pe sistem.
5. **Instalarea utilităților necesare** – Comenzi precum `man`, `pgrep`, `kill`, și `less` sunt utilizate, deci acestea trebuie să fie instalate pe sistemul Linux.